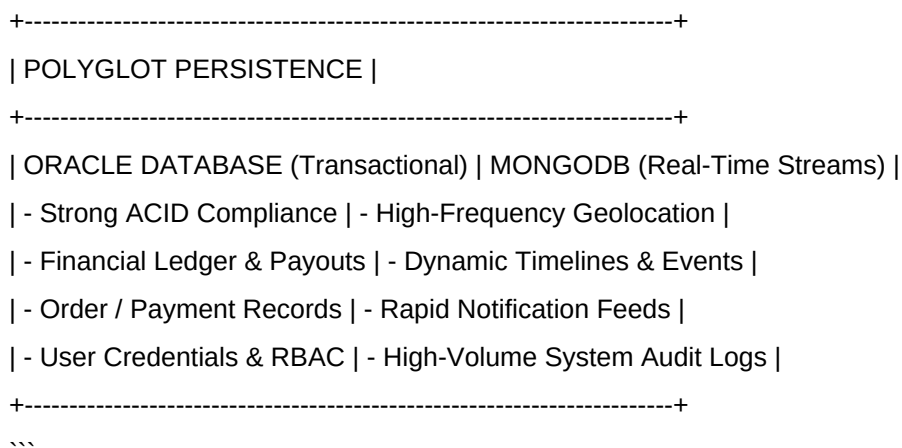


NexFuel Database Architecture & Design Guide

This guide serves as the definitive reference document for the database layer of the NexFuel Fuel Delivery & Logistics Platform. It acts as a blueprint for Spring Boot entity mappings, REST API designs, security auditing, and developer onboarding.

SECTION 1: SYSTEM OVERVIEW

The NexFuel platform leverages a polyglot persistence architecture. Rather than forcing all datasets into a single database technology, we separate concerns between **Oracle Database** (Relational SQL) and **MongoDB** (NoSQL Document Store).



Why Oracle Database Was Chosen

1. **ACID Transactional Guarantees**: Core business operations—such as accepting orders, charging customer credit cards, allocating fuel reserves, and calculating driver payouts—must occur within a strict transactional boundary.
2. **Relational Ledger Integrity**: Relational tables with foreign key constraints prevent orphan records (e.g., payments without orders, settlements without providers).
3. **Advanced Security Controls**: Features like Transparent Data Encryption (TDE) and row-level security audit logs are standard in Oracle and align with enterprise compliance requirements.

Why MongoDB Was Chosen

1. **High-Frequency Geospatial Writes**: Active dispatch drivers transmit GPS coordinates every 5-10 seconds. Writing this high-frequency transient telemetry to a relational database would result in table locking and index bloat. MongoDB handles write-heavy telemetry and supports geospatial index queries natively.
2. **Semi-Structured Document Models**: Live tracking timelines contain dynamic sets of events (e.g., "Departed", "Traffic Delay", "Arrived"). Storing this as a JSON document is more efficient than maintaining deeply nested relational logs.

3. **Horizontal Sharding**: System clickstream activity logs and notifications scale horizontally across document shards without impacting the core relational nodes.

SECTION 2: SYSTEM ARCHITECTURE

The end-to-end data flow separates transactional pathways from high-throughput streaming pipelines:

```
```mermaid
```

```
graph TD
```

```
Client[Client Mobile / Web UI] -->|REST / WebSockets| API[Spring Boot REST Node]
```

```
subgraph Spring Boot Backend Layer
```

```
API -->|JPA / Hibernate| RelationalMgr[Oracle Transaction Manager]
```

```
API -->|MongoTemplate| DocumentMgr[MongoDB Document Manager]
```

```
end
```

```
subgraph Storage Tier
```

```
RelationalMgr -->|SQL Transactions| Oracle[(Oracle Database)]
```

```
DocumentMgr -->|BSON Documents| Mongo[(MongoDB Cluster)]
```

```
end
```

```
Oracle -. ->|Nightly Batch ETL| Mongo
```

```
```
```

End-to-End Data Flow Execution:

1. **Booking Phase**: The customer places a refuel request on the UI. The Spring Boot backend writes an `ORDERS` row (status: `PENDING`) to the Oracle Database.
2. **Allocation Phase**: Spring Boot triggers a geospatial `\$nearSphere` search on MongoDB's `provider\_locations` collection to find the nearest online provider.
3. **Pairing Phase**: Once paired, Spring Boot updates `ORDERS.PROVIDER\_ID` in Oracle and initializes a tracking document in MongoDB's `order\_tracking` collection.
4. **Transit Phase**: As the provider moves, their GPS coordinates are written directly to MongoDB's `provider\_locations` and appended to the path trace inside `order\_tracking`.
5. **Settlement Phase**: Upon delivery, the provider verifies the customer's OTP. The order state transitions in Oracle to `COMPLETED`. Payment is processed, generating rows in Oracle's `PAYMENTS` and `PROVIDER\_SETTLEMENTS` tables.

SECTION 3: ORACLE DATABASE MODULES

The Oracle database schema is organized into 10 logical modules:

...

```
+-----+
| ORACLE DATABASE MODULES |
+-----+
1. AUTHENTICATION	2. CUSTOMER	3. PROVIDER	4. COLLABORATORS
- USERS	- CUST_PROFILES	- PROV_PROFILES	- COLLABORATORS
- ROLES	- VEHICLES	- DOCUMENTS	- FUEL_PRICING
- PERMISSIONS	- AVAILABILITY	- SERVICE_AREAS	
- USER_ROLES	- EARNINGS		
- USER_SESSIONS			
- OTP_VERIFY			
+-----+			
5. ORDERS	6. PAYMENTS	7. FEEDBACK	8. CONTACT CTR
- ORDERS	- PAYMENTS	- FEEDBACK	- INQUIRIES
- ASSIGNMENTS	- TRANSACTIONS	- RESPONSES	
- STATUS_HISTORY	- SETTLEMENTS		
- OTP_VERIFY_LOG	- COMPANY_LEDGER		
+-----+			
9. OPERATIONS	10. ADDITIONAL		
- SERVICE_AREAS	- SERVICE_PACKAGES		
- AUDIT_LOGS	- ORDER_INVOICES		
- SYSTEM_SETTINGS	- NOTIFICATION_SETT		
+-----+
...
```

1. Authentication Module

* **Purpose**: Manages user credentials, role-based access control (RBAC), and active session indexes.

* **Business Usage**: Authenticates logins, enforces customer vs. provider role partitions, and manages password reset OTP flows.

* **Relationships**: `USERS` maps M:N to `ROLES` via `USER\_ROLES` and has a 1:N relationship with `USER\_SESSIONS` and `OTP\_VERIFICATIONS`.

2. Customer Module

* **Purpose**: Manages customer profiles, vehicle registrations, and default delivery coordinates.

* **Business Usage**: Retains customer ratings and profiles, and stores vehicle details (make, model, license plates, fuel type) to assist providers on-scene.

* **Relationships**: `CUSTOMER\_PROFILES` has a 1:1 relationship with `USERS` and a 1:N relationship with `VEHICLES` and `ORDERS`.

3. Provider Module

* **Purpose**: Manages fleet driver profiles, commercial license documents, availability state tracking, and earnings accounts.

* **Business Usage**: Verifies CDL/Hazmat files, tracks driver statuses (Active/Suspended/Removed), and aggregates provider payouts.

* **Relationships**: `PROVIDER\_PROFILES` has a 1:1 relationship with `USERS` and 1:N relationships with `PROVIDER\_DOCUMENTS`, `PROVIDER\_AVAILABILITY`, and `PROVIDER\_SETTLEMENTS`.

4. Collaborators Module

* **Purpose**: Manages retail petrol bunk partnership depots and fuel pricing sheets.

* **Business Usage**: Monitors retail depots where tankers reload, tracks fuel costs per gallon, and manages service areas for partner stations.

* **Relationships**: `COLLABORATORS` maps M:N to `SERVICE\_AREAS` (via `COLLABORATOR\_SERVICE\_AREAS`) and 1:N to `COLLABORATOR\_FUEL\_PRICING`.

5. Orders Module

* **Purpose**: Manages transactional dispatches, driver pairing logs, and historical status audits.

* **Business Usage**: Records order details (fuel grade, volume, charges), driver allocations, and coordinates target coordinates.

* **Relationships**: `ORDERS` has 1:N relationships with `ORDER\_ASSIGNMENTS` and `ORDER\_STATUS\_HISTORY`, and a 1:1 relationship with `PAYMENTS`.

6. Payments Module

* **Purpose**: Manages payment transactions, gateway refs, provider payout logs, and company ledgers.

* **Business Usage**: Charges customers, records credit card/UPI transactions, tracks corporate commissions, and logs payouts.

* **Relationships**: `PAYMENTS` has a 1:N relationship with `PAYMENT\_TRANSACTIONS` and `PROVIDER\_SETTLEMENTS` (payout disbursements).

7. Feedback Module

* **Purpose**: Logs customer ratings and reviews, along with support team responses.

* **Business Usage**: Analyzes ratings, category complaints, and recommendations, and tracks escalated issues.

* **Relationships**: `FEEDBACK` references `ORDERS` and maps 1:N to `FEEDBACK\_RESPONSES`.

8. Contact Center Module

* **Purpose**: Logs public inquiries, partnership proposals, and collaboration requests.

* **Business Usage**: Tracks contact center inquiries, filters priorities, and manages the conversion of bunk proposals to partner accounts.

* **Relationships**: Independent table (`INQUIRIES`) that can convert rows into new `COLLABORATORS` records.

9. Operations Module

* **Purpose**: Manages service area zones, audit logs, and global configuration values.

* **Business Usage**: Evaluates coordinate polygons for city zones, retains administrative clickstream histories, and stores system settings.

* **Relationships**: Lookup tables referenced across modules.

10. Additional Module

* **Purpose**: Manages service packages (standard/rapid tiers), invoice logs, and notification configurations.

* **Business Usage**: Configures base service rates, stores generated invoices, and manages notification settings.

* **Relationships**: Extends `ORDERS`, `USERS`, and `PAYMENTS` with auxiliary parameters.

SECTION 4: TABLE DOCUMENTATION

Below is the column metadata, constraint mapping, and example records for all tables:

1. USERS

* **Purpose**: Core login credentials and status records.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented via `SEQ_USERS`.

* `FULL_NAME` (VARCHAR2(100)): Full name.

* `EMAIL` (VARCHAR2(150), Unique): Email address.

* `PHONE_NUMBER` (VARCHAR2(30), Nullable): Phone number.

* `PASSWORD_HASH` (VARCHAR2(255)): BCrypt password hash.

* `STATUS` (VARCHAR2(20)): Status (Check: `'ACTIVE'`, `'INACTIVE'`, `'SUSPENDED'`).

* `CREATED_AT` / `UPDATED_AT` (TIMESTAMP): Timestamps.

* **Indexes**: PK Index on `ID`, Unique Index on `EMAIL`.

* **Example Record**:

`ID`: 1000, `FULL_NAME`: "John Doe", `EMAIL`: "john@nexfuel.com", `PHONE_NUMBER`: "+919876543210", `PASSWORD_HASH`: "\$2a\$12\$V.8G...", `STATUS`: "ACTIVE"

2. ROLES

* **Purpose**: Enforces access permissions.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented via `SEQ_ROLES`.

- * `NAME` (VARCHAR2(50), Unique): Role name.
- * `DESCRIPTION` (VARCHAR2(255)): Role details.
- * **Example Record**:
`ID: 1, NAME: "ROLE\_CUSTOMER", DESCRIPTION: "Standard fuel purchaser account"`

3. PERMISSIONS

- * **Purpose**: Enforces functional permission keys.
- * **Columns**:
 - * `ID` (NUMBER(19), PK): Autoincremented.
 - * `NAME` (VARCHAR2(100), Unique): Action permissions.
 - * `DESCRIPTION` (VARCHAR2(255)): Permission description.
- * **Example Record**:
`ID: 1, NAME: "WRITE\_DISPATCH\_REQUEST", DESCRIPTION: "Ability to request fuel dispatch"`

4. USER_ROLES

- * **Purpose**: Join table linking users to roles.
- * **Columns**:
 - * `USER\_ID` (NUMBER(19), FK): References `USERS.ID`.
 - * `ROLE\_ID` (NUMBER(19), FK): References `ROLES.ID`.
- * **Constraints**: Composite Primary Key (`USER\_ID`, `ROLE\_ID`).

5. ROLE_PERMISSIONS

- * **Purpose**: Join table linking roles to permission tags.
- * **Columns**:
 - * `ROLE\_ID` (NUMBER(19), FK): References `ROLES.ID`.
 - * `PERMISSION\_ID` (NUMBER(19), FK): References `PERMISSIONS.ID`.
- * **Constraints**: Composite Primary Key (`ROLE\_ID`, `PERMISSION\_ID`).

6. USER_SESSIONS

- * **Purpose**: Tracks active session tokens.
- * **Columns**:
 - * `ID` (NUMBER(19), PK): Autoincremented.
 - * `USER\_ID` (NUMBER(19), FK): References `USERS.ID`.
 - * `TOKEN` (VARCHAR2(500)): Cryptographic session token.
 - * `EXPIRED\_AT` (TIMESTAMP): Expiration date.
 - * `STATUS` (VARCHAR2(20)): Active state (Check: `ACTIVE`, `EXPIRED`, `REVOKED`).
 - * `IP\_ADDRESS` / `DEVICE\_INFO` (VARCHAR2): Client environment parameters.
- * **Example Record**:

`ID: 1, USER\_ID: 1000, TOKEN: "eyJhbGc...", EXPIRED\_AT: 2026-06-17, STATUS: "ACTIVE"`

7. OTP_VERIFICATIONS

* **Purpose**: Tracks single-use security tokens.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `USER\_ID` (NUMBER(19), FK): References `USERS.ID`.

* `OTP\_CODE` (VARCHAR2(10)): SHA-256 OTP code.

* `VERIFY\_TYPE` (VARCHAR2(20)): (Check: `EMAIL\_SIGNUP`, `PASSWORD\_RESET`, `PHONE\_VERIFY`).

* `EXPIRED\_AT` (TIMESTAMP): Expiration timestamp.

* `ATTEMPTS` (NUMBER(3)): Attempt count.

* `STATUS` (VARCHAR2(20)): (Check: `PENDING`, `VERIFIED`, `EXPIRED`).

* **Example Record**:

`ID: 1, USER\_ID: 1000, OTP\_CODE: "89afc...", VERIFY\_TYPE: "EMAIL\_SIGNUP", ATTEMPTS: 0, STATUS: "PENDING"`

8. CUSTOMER_PROFILES

* **Purpose**: Stores customer metadata.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `USER\_ID` (NUMBER(19), FK, Unique): References `USERS.ID`.

* `RATING` (NUMBER(3,2)): Average customer score.

* `DEFAULT\_ADDRESS` (VARCHAR2(255)): Standard delivery address.

* **Example Record**:

`ID: 100, USER\_ID: 1000, RATING: 4.85, DEFAULT\_ADDRESS: "Koramangala 4th block, Bangalore"`

9. VEHICLES

* **Purpose**: Customer vehicle specifications.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `CUSTOMER\_ID` (NUMBER(19), FK): References `CUSTOMER\_PROFILES.ID`.

* `MAKE` / `MODEL` / `YEAR\_VAL` (VARCHAR2/NUMBER): Vehicle specifications.

* `LICENSE\_PLATE` (VARCHAR2(30), Unique): License plate.

* `FUEL\_TYPE` (VARCHAR2(30)): Fuel grade (Check: `87 Regular`, `91 Premium`, `Diesel`).

* **Example Record**:

`ID: 100, CUSTOMER\_ID: 100, MAKE: "Ford", MODEL: "F-150", YEAR\_VAL: 2024, LICENSE\_PLATE: "KA-01-MJ-9999", FUEL\_TYPE: "91 Premium"`

10. PROVIDER_PROFILES

* **Purpose**: Stores active fleet driver statistics.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `USER\_ID` (NUMBER(19), FK, Unique): References `USERS.ID`.

* `STATUS` (VARCHAR2(20)): Status (Check: `PENDING`, `ACTIVE`, `SUSPENDED`, `REJECTED`, `REMOVED`).

* `RATING` (NUMBER(3,2)): Driver rating.

* `COMPLETED\_DELIVERIES` (NUMBER(9)): Total completed trips.

* **Example Record**:

`ID: 100, USER\_ID: 1005, STATUS: "ACTIVE", RATING: 4.95, COMPLETED\_DELIVERIES: 142`

11. PROVIDER_DOCUMENTS

* **Purpose**: Safety licenses (CDL, Hazmat).

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `PROVIDER\_ID` (NUMBER(19), FK): References `PROVIDER\_PROFILES.ID`.

* `DOC\_TYPE` (VARCHAR2(50)): Document category (e.g. `CDL`).

* `DOC\_NAME` (VARCHAR2(100)): File name.

* `FILE\_URL` (VARCHAR2(500)): Cloud file URL.

* `STATUS` (VARCHAR2(20)): (Check: `PENDING`, `APPROVED`, `REJECTED`, `EXPIRED`).

* `EXPIRES\_AT` (DATE): Expiration date.

* **Example Record**:

`ID: 100, PROVIDER\_ID: 100, DOC\_TYPE: "HAZMAT", DOC\_NAME: "hazmat\_cert.pdf", STATUS: "APPROVED", EXPIRES\_AT: 2026-12-31`

12. PROVIDER_AVAILABILITY

* **Purpose**: Logs online status sessions.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `PROVIDER\_ID` (NUMBER(19), FK): References `PROVIDER\_PROFILES.ID`.

* `STATUS` (VARCHAR2(20)): Availability (Check: `AVAILABLE`, `UNAVAILABLE`, `ON\_JOB`).

* `LAST\_ACTIVE\_AT` (TIMESTAMP): Last ping update.

* **Example Record**:

`ID: 1, PROVIDER\_ID: 100, STATUS: "AVAILABLE", LAST\_ACTIVE\_AT: 2026-06-16 15:00:00`

13. PROVIDER_EARNINGS

* **Purpose**: Tracks provider earnings.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `PROVIDER\_ID` (NUMBER(19), FK, Unique): References `PROVIDER\_PROFILES.ID`.

* `GROSS\_AMOUNT` / `PAID\_AMOUNT` / `PENDING\_AMOUNT` (NUMBER(12,2)): Earnings ledger amounts.

* `LAST\_UPDATED\_AT` (TIMESTAMP): Last update timestamp.

* **Example Record**:

`ID`: 1, `PROVIDER\_ID`: 100, `GROSS\_AMOUNT`: 1240.00, `PAID\_AMOUNT`: 1000.00,
`PENDING\_AMOUNT`: 240.00`

14. COLLABORATORS

* **Purpose**:

 Stores partner bunk profile details.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `NAME` (VARCHAR2(150)): Bunk name.

* `CONTACT\_PERSON` / `EMAIL` / `PHONE\_NUMBER` / `ADDRESS` (VARCHAR2): Partner details.

* `STATUS` (VARCHAR2(20)): (Check: `PROSPECT`, `ACTIVE\_PARTNER`, `SUSPENDED`, `INACTIVE`).

* `CONTRACT\_EXPIRY\_DATE` (DATE): Contract expiration date.

* `NOTES` (VARCHAR2(1000)): Internal contract notes.

* **Example Record**:

`ID`: 10, `NAME`: "HP Station", `CONTACT\_PERSON`: "Ramesh Kumar", `EMAIL`: "ramesh@hp.co.in",
`STATUS`: "ACTIVE_PARTNER", `CONTRACT\_EXPIRY\_DATE`: 2026-12-15`

15. COLLABORATOR_FUEL_PRICING

* **Purpose**:

 Stores partner bunk fuel price lists.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `COLLABORATOR\_ID` (NUMBER(19), FK): References `COLLABORATORS.ID`.

* `FUEL\_TYPE` (VARCHAR2(30)): Fuel grade (Check: `87 Regular`, `91 Premium`, `Diesel`).

* `PRICE` (NUMBER(5,2)): Cost per gallon.

* `LAST\_UPDATED\_AT` (TIMESTAMP): Last price update timestamp.

* **Example Record**:

`ID`: 1, `COLLABORATOR\_ID`: 10, `FUEL\_TYPE`: "91 Premium", `PRICE`: 4.15`

16. COLLABORATOR_SERVICE_AREAS

* **Purpose**:

 Join table mapping collaborators to service areas.

* **Columns**:

* `COLLABORATOR\_ID` (NUMBER(19), FK): References `COLLABORATORS.ID`.

* `SERVICE\_AREA\_ID` (NUMBER(19), FK): References `SERVICE\_AREAS.ID`.

17. ORDERS

* **Purpose**: Stores core order transaction details.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `CUSTOMER\_ID` (NUMBER(19), FK): References `CUSTOMER\_PROFILES.ID`.

* `VEHICLE\_ID` (NUMBER(19), FK): References `VEHICLES.ID`.

* `PROVIDER\_ID` (NUMBER(19), FK, Nullable): References `PROVIDER\_PROFILES.ID`.

* `STATUS` (VARCHAR2(20)): Order status (Check: `PENDING`, `ACTIVE`, `DELIVERED`, `COMPLETED`, `CANCELLED`).

* `FUEL\_TYPE` (VARCHAR2(30)): Fuel grade (Check: `87 Regular`, `91 Premium`, `Diesel`).

* `QUANTITY\_GALLONS` (NUMBER(6,2)): Quantity.

* `DELIVERY\_CHARGE` / `FUEL\_COST` / `TOTAL\_AMOUNT` (NUMBER): Cost details.

* `TARGET\_LATITUDE` / `TARGET\_LONGITUDE` (NUMBER(9,6)): Delivery coordinates.

* `TARGET\_ADDRESS` (VARCHAR2(255)): Delivery address.

* **Example Record**:

`ID`: 5000, `CUSTOMER\_ID`: 100, `VEHICLE\_ID`: 100, `PROVIDER\_ID`: 100, `STATUS`: "COMPLETED", `FUEL\_TYPE`: "91 Premium", `QUANTITY\_GALLONS`: 5.00, `TOTAL\_AMOUNT`: 39.25, `TARGET\_LATITUDE`: 42.3601, `TARGET\_LONGITUDE`: -71.0589`

18. ORDER_ASSIGNMENTS

* **Purpose**: Logs driver match routing iterations.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `ORDER\_ID` (NUMBER(19), FK): References `ORDERS.ID`.

* `PROVIDER\_ID` (NUMBER(19), FK): References `PROVIDER\_PROFILES.ID`.

* `STATUS` (VARCHAR2(20)): (Check: `PENDING`, `ACCEPTED`, `REJECTED`, `EXPIRED`).

* `ASSIGNED\_AT` / `RESPONDED\_AT` (TIMESTAMP): Allocation timestamps.

19. ORDER_STATUS_HISTORY

* **Purpose**: Order milestone audit logs.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `ORDER\_ID` (NUMBER(19), FK): References `ORDERS.ID`.

* `STATUS` (VARCHAR2(20)): Status milestone.

* `CHANGED\_BY` (VARCHAR2(50)): Actor identifier (e.g. `PROVIDER`).

* `NOTES` (VARCHAR2(255)): Status notes.

20. ORDER_OTP_VERIFICATIONS

* **Purpose**: Logs OTP verification handshakes.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `ORDER\_ID` (NUMBER(19), FK, Unique): References `ORDERS.ID`.

* `OTP\_HASH` (VARCHAR2(255)): Handshake verification hash.

* `IS\_VERIFIED` (NUMBER(1)): (Check: `0` or `1`).

21. PAYMENTS

* **Purpose**: Billing invoice transaction header.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `ORDER\_ID` (NUMBER(19), FK, Unique): References `ORDERS.ID`.

* `AMOUNT` (NUMBER(8,2)): Ledger amount.

* `STATUS` (VARCHAR2(20)): Payment status (Check: `PENDING`, `SUCCESSFUL`, `FAILED`).

* `GATEWAY` (VARCHAR2(30)): (e.g. `RAZORPAY`).

* `PAYMENT\_METHOD` (VARCHAR2(30)): Payment channel (e.g. `CREDIT\_CARD`).

22. PAYMENT_TRANSACTIONS

* **Purpose**: Logs payment gateway transaction events.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `PAYMENT\_ID` (NUMBER(19), FK): References `PAYMENTS.ID`.

* `TRANSACTION\_REF` (VARCHAR2(150), Unique): Payment gateway reference.

* `GATEWAY\_RESPONSE` (CLOB): Gateway response details.

* `STATUS` (VARCHAR2(20)): (Check: `INITIATED`, `SUCCESSFUL`, `FAILED`, `REFUNDED`).

23. PROVIDER_SETTLEMENTS

* **Purpose**: Logs driver payout settlements.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `PROVIDER\_ID` (NUMBER(19), FK): References `PROVIDER\_PROFILES.ID`.

* `AMOUNT` (NUMBER(10,2)): Settlement payout amount.

* `PAYMENT\_DATE` (DATE): Payment date.

* `TRANSACTION\_REF` (VARCHAR2(150), Unique): Payment reference.

* `STATUS` (VARCHAR2(20)): (Check: `PROCESSED`, `FAILED`, `HELD`).

24. COMPANY_LEDGER

* **Purpose**: Logs corporate revenue transactions.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `ORDER\_ID` (NUMBER(19), FK): References `ORDERS.ID`.

* `GROSS\_REVENUE` / `PROVIDER\_PAYOUT` / `COMPANY\_COMMISSION` (NUMBER(10,2)): Revenue breakdown.

* `CREATED\_AT` (TIMESTAMP): Creation timestamp.

25. FEEDBACK

* **Purpose**: Stores customer ratings and reviews.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `ORDER\_ID` (NUMBER(19), FK): References `ORDERS.ID`.

* `CUSTOMER\_ID` (NUMBER(19), FK): References `CUSTOMER\_PROFILES.ID`.

* `PROVIDER\_ID` (NUMBER(19), FK): References `PROVIDER\_PROFILES.ID`.

* `RATING` (NUMBER(1)): (Check: `1` to `5`).

* `CATEGORY` (VARCHAR2(50)): Feedback category.

* `RECOMMEND` (VARCHAR2(10)): Recommend status (Check: `Yes`, `No`).

* `CONTACT\_PERMISSION` (NUMBER(1)): Contact permission (Check: `0` or `1`).

* `COMMENT\_TEXT` (VARCHAR2(1000)): Customer feedback comment.

* `STATUS` (VARCHAR2(20)): Review status (Check: `New`, `Reviewed`, `Escalated`, `Resolved`).

26. FEEDBACK_RESPONSES

* **Purpose**: Logs support responses to feedback.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `FEEDBACK\_ID` (NUMBER(19), FK): References `FEEDBACK.ID`.

* `ADMIN\_USER\_ID` (NUMBER(19), FK): References `USERS.ID`.

* `RESPONSE\_TEXT` (VARCHAR2(1000)): Support reply text.

27. INQUIRIES

* **Purpose**: Logs contact center inquiries.

* **Columns**:

* `ID` (NUMBER(19), PK): Autoincremented.

* `FULL\_NAME` / `EMAIL` / `PHONE\_NUMBER` / `SUBJECT` / `DESCRIPTION` (VARCHAR2/CLOB): Inquirer profile details.

* `PREFERRED\_CONTACT` (VARCHAR2(20)): (Check: `EMAIL`, `PHONE`).

- * `CATEGORY` (VARCHAR2(50)): Inquiry category.
- * `COMPANY\_NAME` / `BUSINESS\_TYPE` / `COMPANY\_WEBSITE` / `SERVICE\_REGION` (VARCHAR2): Dynamic business details.
- * `STATUS` (VARCHAR2(20)): Inquiry status (Check: `New`, `In Review`, `Contacted`, `Accepted`, `Rejected`, `Closed`).
- * `LEAD\_PRIORITY` (VARCHAR2(20)): Priority (Check: `Low`, `Medium`, `High`, `Critical`).
- * `LEAD\_SOURCE` (VARCHAR2(50)): Lead source (e.g. `Website Contact`).

28. SERVICE_AREAS

- * **Purpose**: Stores lookup metadata for service zones.
- * **Columns**:
 - * `ID` (NUMBER(19), PK): Autoincremented.
 - * `CITY` / `NAME` (VARCHAR2): Zone name.
 - * `STATUS` (VARCHAR2(20)): Active state (Check: `ACTIVE`, `INACTIVE`).
 - * `POLYGON\_COORDS` (CLOB): GeoJSON service area polygon definition.

29. AUDIT_LOGS

- * **Purpose**: Logs administrative clickstream audit trails.
- * **Columns**:
 - * `ID` (NUMBER(19), PK): Autoincremented.
 - * `USER\_ID` (NUMBER(19), FK, Nullable): References `USERS.ID`.
 - * `ACTION` / `TARGET\_TABLE` / `DETAILS` / `IP\_ADDRESS` (VARCHAR2): Audit details.
 - * `RECORD\_ID` (NUMBER(19)): Audited row record ID.

30. SYSTEM_SETTINGS

- * **Purpose**: Global platform parameters.
- * **Columns**:
 - * `CONFIG\_KEY` (VARCHAR2(100), PK): System property key.
 - * `CONFIG\_VALUE` (VARCHAR2(500)): Property value.
 - * `DESCRIPTION` (VARCHAR2(255)): Property details.

31. SERVICE_PACKAGES

- * **Purpose**: Platform service tier rates configuration.
- * **Columns**:
 - * `ID` (NUMBER(19), PK): Autoincremented.
 - * `NAME` (VARCHAR2(50)): Tier name (e.g. `Rapid Delivery`).
 - * `BASE\_PRICE` (NUMBER(6,2)): Dispatch fee.
 - * `ESTIMATED\_ETA\_MINS` (NUMBER(3)): ETA wait minutes.

32. ORDER_INVOICES

- * **Purpose**: Order invoice details.
- * **Columns**:
 - * `ID` (NUMBER(19), PK): Autoincremented.
 - * `ORDER\_ID` (NUMBER(19), FK, Unique): References `ORDERS.ID`.
 - * `INVOICE\_NUMBER` (VARCHAR2(100), Unique): Invoice identification.
 - * `PDF\_FILE\_URL` (VARCHAR2(500)): PDF file URL.

33. USER_NOTIFICATION_SETTINGS

- * **Purpose**: Manages user notification configurations.
- * **Columns**:
 - * `USER\_ID` (NUMBER(19), PK, FK): References `USERS.ID`.
 - * `PUSH\_ENABLED` / `EMAIL\_ENABLED` (NUMBER(1)): (Check: `0` or `1`).

SECTION 5: ENTITY RELATIONSHIPS

The entity relationships are configured as follows:

```

'''
+-----+
| USERS |
+-----+
|
+-----+-----+
| 1:1 | 1:1
+-----+ +-----+
| CUSTOMER_PROFILES || PROVIDER_PROFILES |
+-----+ +-----+
||
| 1:N | 1:N
+-----+ +-----+
| VEHICLES || PROVIDER_DOCUMENTS|
+-----+ +-----+
||
+-----+-----+
| 1:N
+-----+
```

```

| ORDERS |
+-----+
| 1:1
+-----+
| PAYMENTS |
+-----+
| 1:N
+-----+
|PAYMENT_TRANSACTION|
+-----+
...

```

1. One-To-One (1:1) Relationships

* **`USERS` <-> `CUSTOMER\_PROFILES`**:

Relational customers have a single customer profile row containing addresses and ratings.

* **`USERS` <-> `PROVIDER\_PROFILES`**:

Operational refueler providers have a single profile row containing status parameters.

* **`ORDERS` <-> `PAYMENTS`**:

An order dispatch generates a single payment invoice header upon delivery.

2. One-To-Many (1:N) Relationships

* **`CUSTOMER\_PROFILES` -> `VEHICLES`**:

A customer can register multiple vehicles (e.g. Ford F-150, Tesla Model X) to their profile.

* **`CUSTOMER\_PROFILES` -> `ORDERS`**:

A customer can place multiple fuel requests over time.

* **`PROVIDER\_PROFILES` -> `PROVIDER\_DOCUMENTS`**:

A provider uploads multiple files (CDL, Hazmat permit, insurance policies) for verification.

* **`ORDERS` -> `ORDER\_ASSIGNMENTS`**:

An order dispatch can be routed and matched to multiple drivers sequentially if previous attempts time out.

* **`PAYMENTS` -> `PAYMENT\_TRANSACTIONS`**:

A payment invoice can have multiple gateway payment attempts (initiated, failed, successful).

3. Many-To-Many (M:N) Relationships

* **`USERS` <-> `ROLES` (via `USER\_ROLES`)**:

Multiple users can share the same role (e.g. `ROLE\_CUSTOMER`), and a user can have multiple roles (e.g. customer and provider).

* **`COLLABORATORS` <-> `SERVICE\_AREAS` (via `COLLABORATOR\_SERVICE\_AREAS`)**:

Bunks can supply fuel across multiple regions, and service areas can draw from multiple partner stations.

SECTION 6: MONGODB COLLECTIONS

The MongoDB collection schemas handle high-frequency coordinates and transient data:

1. provider_locations

* **Purpose**: Tracks active provider GPS coordinates.

* **Index Strategy**: `2dsphere` geospatial index on the `location` field.

* **Document Schema**:

```
```json
{
 "_id": {"$oid": "603f9e925bf0fc208c0234a1"},
 "provider_id": 100,
 "provider_name": "Marcus Vance",
 "location": {
 "type": "Point",
 "coordinates": [-71.0589, 42.3601]
 },
 "is_online": true,
 "availability_status": "AVAILABLE",
 "last_updated_at": {"$date": "2026-06-16T15:30:00.000Z"}
}
```

\* \*\*Business Usage\*\*: Supports the `\$nearSphere` search queries used to pair dispatchers with customer orders.

## 2. order\_tracking

\* \*\*Purpose\*\*: Tracks live order dispatch status histories and GPS transit coordinate path traces.

\* \*\*Index Strategy\*\*: Hashed index on `order\_id` for quick matching checks.

\* \*\*Document Schema\*\*:

```
```json
{
  "_id": {"$oid": "603f9e925bf0fc208c0234b2"},
  "order_id": 5000,
  "customer_id": 100,
  "provider_id": 100,
  "timeline": [
    { "event": "Order Placed", "status": "PENDING", "timestamp": {"$date": "2026-06-16T15:00:00.000Z"} },
    { "event": "Provider Dispatched", "status": "ACTIVE", "timestamp": {"$date": "2026-06-16T15:02:15.000Z"} }
  ],
}
```



```
"gps_path_history": [
  { "coordinates": [-71.0504, 42.3551], "timestamp": { "$date": "2026-06-16T15:03:00.000Z" } }
]
}
...

```

* **Business Usage**: Feeds the customer tracking page with real-time ETA updates and map traces.

3. notifications

* **Purpose**: Stores read/unread notifications.

* **Index Strategy**: Compound index on `{ user\_id: 1, is\_read: 1, created\_at: -1 }`.

* **Document Schema**:

```
```json
{
 "_id": { "$oid": "603f9e925bf0fc208c0234c3" },
 "user_id": 1000,
 "title": "Refueler En Route",
 "message": "Your driver Marcus is en route. ETA: 12 mins.",
 "is_read": false,
 "created_at": { "$date": "2026-06-16T15:02:20.000Z" }
}
...

```

\* \*\*Business Usage\*\*: Powers the in-app notification center.

### 4. analytics\_snapshots

\* \*\*Purpose\*\*: Stores daily aggregated system performance data.

\* \*\*Index Strategy\*\*: Hashed index on `snapshot\_date`.

\* \*\*Document Schema\*\*:

```
```json
{
  "_id": { "$oid": "603f9e925bf0fc208c0234d4" },
  "snapshot_date": "2026-06-16",
  "revenue": { "gross_total": 48920.50, "payouts": 39136.40 },
  "dispatches": { "total_orders": 124, "completed": 120 }
}
...

```

* **Business Usage**: Powers administrative dashboards and analytics reports.

5. system_activity_logs

* **Purpose***: Captures user session clickstream histories and system activity logs.

* **Index Strategy***: Hashed sharding index on `timestamp`.

* **Document Schema**:

```
``json
{
  "_id": {"$oid": "603f9e925bf0fc208c0234e5"},
  "user_id": 1000,
  "role": "customer",
  "activity_type": "GEO_LOCATION_UPDATE",
  "details": { "page": "/fuel-request", "coords": [-71.0504, 42.3551] }
}
```

* **Business Usage***: Supports security auditing and platform telemetry analysis.

SECTION 7: BUSINESS WORKFLOW MAPPING

Here is how data flows through Oracle Database and MongoDB during core business operations:

[CUSTOMER REGISTRATION]

1. UI submits credentials -> API computes BCrypt hash.
2. Insert USERS (Oracle) -> Insert CUSTOMER_PROFILES (Oracle) -> Insert USER_NOTIFICATION_SETTINGS (Oracle).
3. Insert verification OTP_VERIFICATIONS (Oracle) -> Send welcome verification email.
4. Customer verifies OTP -> Update OTP_VERIFICATIONS & USERS state to ACTIVE (Oracle).

[FUEL DISPATCH REQUEST]

1. Customer submits fuel request from UI -> API verifies token in USER_SESSIONS (Oracle).
2. Insert ORDERS row status 'PENDING' (Oracle) -> Write ORDER_STATUS_HISTORY row (Oracle).
3. Query MongoDB provider_locations using geospatial '\$nearSphere' index to locate nearest driver.
4. Create ORDER_ASSIGNMENTS row (Oracle) -> Set assignment status to 'PENDING'.
5. Send push notification to Provider -> Write notifications (MongoDB).
6. Driver accepts -> Update ORDER_ASSIGNMENTS & ORDERS status to 'ACTIVE' (Oracle).
7. Create tracking document in order_tracking (MongoDB) -> Populate timeline event 'Driver Dispatched'.

1. Payment Settlement Workflow

1. Driver arrives -> verifies customer OTP -> updates `ORDERS.STATUS` to `DELIVERED` in Oracle.
2. Timeline event `Delivered` is appended to `order\_tracking` in MongoDB.
3. Customer triggers checkout. Spring Boot hits the Razorpay API.
4. Create `PAYMENTS` (status `PENDING`) and `PAYMENT\_TRANSACTIONS` (status `INITIATED`) in Oracle.
5. Razorpay returns Success payload -> Update `PAYMENTS` and `PAYMENT\_TRANSACTIONS` to `SUCCESSFUL` / `SUCCESSFUL` in Oracle.
6. Trigger ledger distribution: Calculate 20% corp fee and 80% driver fee -> Insert details to `COMPANY\_LEDGER` (Oracle) and increment pending balance in `PROVIDER\_EARNINGS` (Oracle).
7. Update `ORDERS.STATUS` to `COMPLETED` in Oracle.

2. Feedback Logging Workflow

1. Order completes -> customer triggers rating review.
2. Insert review data to `FEEDBACK` in Oracle (status `New`).
3. If Star rating & 2 Stars, set `FEEDBACK.STATUS` to `Escalated` in Oracle.
4. Trigger average rating recalculation -> Update `PROVIDER\_PROFILES.RATING` and `PROVIDER\_PROFILES.COMPLETED\_DELIVERIES` (increment by 1) in Oracle.

SECTION 8: BUSINESS RULES

To ensure platform stability, the database layer enforces the following validation checks:

- * **Payment Cleared Constraint**: Orders cannot transition to `COMPLETED` status in Oracle unless the corresponding `PAYMENTS.STATUS` is `SUCCESSFUL`.
- * **Rider Matching Eligibility**: Providers must be online (`is\_online: true` in MongoDB) and available (status: `AVAILABLE` in Oracle and MongoDB) to receive order assignments.
- * **Feedback Restrictions**: Customers can only submit reviews when the referenced order has status `COMPLETED` and payment is successfully settled.
- * **Depot Compliance**: A collaborator station can only supply fuel if its status is `ACTIVE\_PARTNER` and its contract expiration date is in the future.
- * **Driver Document Compliance**: A provider cannot go online if any of their verified document records (`PROVIDER\_DOCUMENTS`) have expired or been rejected.

SECTION 9: STATE MACHINE DIAGRAMS

The database layer tracks several key entity lifecycles:

1. Order Lifecycle

```
```mermaid
stateDiagram-v2
[*] --> PENDING : Customer Request
PENDING --> ASSIGNED : Provider Found
ASSIGNED --> ACCEPTED : Provider Accepts
ACCEPTED --> EN_ROUTE : Transit Starts
EN_ROUTE --> DELIVERED : Fuel Pumped
DELIVERED --> OTP_VERIFIED : Handshake Match
OTP_VERIFIED --> PAYMENT_PENDING : PG Checkout
PAYMENT_PENDING --> COMPLETED : Ledger Settled
PENDING --> CANCELLED : Timeout / Reject
```
```

2. Provider Lifecycle

```
```mermaid
stateDiagram-v2
[*] --> PENDING : Document Upload
PENDING --> VERIFIED : Audits Approved
VERIFIED --> AVAILABLE : Toggle Online
AVAILABLE --> ON_JOB : Accept Request
ON_JOB --> AVAILABLE : Service Finished
AVAILABLE --> UNAVAILABLE : Toggle Offline
VERIFIED --> SUSPENDED : Admin Flagged
```
```

3. Payment Lifecycle

```
```mermaid
stateDiagram-v2
[*] --> PENDING : Invoice Generated
PENDING --> INITIATED : PG Handshake
INITIATED --> SUCCESSFUL : Ledger Cleared
INITIATED --> FAILED : Card Declined
```
```

4. Collaborator Lifecycle

```
```mermaid
stateDiagram-v2
```

[\*] --> PROSPECT : Proposal Received  
PROSPECT --> ACTIVE\_PARTNER : Contract Signed  
ACTIVE\_PARTNER --> SUSPENDED : Audit Flagged  
ACTIVE\_PARTNER --> INACTIVE : Contract Expired  
...  
  
---

## SECTION 10: DATABASE SECURITY

To secure user and transaction data, the system implements the following database-level protocols:

- \* **Password Security**: Passwords are hashed using BCrypt (strength 12) before being stored in ``USERS.PASSWORD_HASH``.
- \* **OTP Security**: OTP codes are stored as SHA-256 hashes inside ``OTP_VERIFICATIONS.OTP_CODE`` to prevent plain text exposure in DB dumps.
- \* **Session Validation**: API nodes validate incoming JWTs against the ``USER_SESSIONS`` table. If a session is marked ``REVOKED``, the request is rejected immediately.
- \* **Encryption at Rest**: Sensitive columns (e.g., customer/provider emails, phone numbers, license plates) are encrypted using Oracle Transparent Data Encryption (TDE).
- \* **Role-Based Access Control (RBAC)**: Fine-grained database access is enforced via role-based mappings (``USER_ROLES`` and ``ROLE_PERMISSIONS``).

---

## SECTION 11: PERFORMANCE DESIGN

The database is optimized to handle high-frequency transactions and spatial queries:

### 1. Oracle Indexing Strategy

- \* B-Tree indexes are created on *all* foreign key columns (e.g. ``IX_ORD_CUST``, ``IX_ORD_PROV``) to prevent parent-table locking on deletes.
- \* Composite indexes are created on common query fields:
  - ``IX_ORDERS_STATUS`` (STATUS, CREATED\_AT DESC)
  - ``IX_FEED_STATUS_RATING`` (STATUS, RATING DESC)

### 2. Oracle Partitioning Strategy

- \* Range partitioning is applied to ``AUDIT_LOGS`` and ``ORDER_STATUS_HISTORY`` tables by date (``CREATED_AT``) to facilitate partition pruning and historical log archiving.
- \* List partitioning is applied to ``ORDERS`` by ``STATUS`` to optimize dashboard filters.

### 3. MongoDB Geospatial Queries

- \* A ``2dsphere`` index is configured on the ``location`` field in the ``provider_locations`` collection.

\* Spring Boot queries active drivers using a geospatial `\$nearSphere` query:

```
```json
{
  "location": {
    "$nearSphere": {
      "$geometry": {
        "type": "Point",
        "coordinates": [-71.0589, 42.3601]
      },
      "$maxDistance": 8046.72
    }
  },
  "is_online": true,
  "availability_status": "AVAILABLE"
}
```

```

## SECTION 12: DATABASE NAMING STANDARDS

To maintain design consistency across development teams, all database objects must follow these naming conventions:

\* **Tables**: `UPPER\_CASE` singular/plural nouns (e.g. `USERS`, `CUSTOMER\_PROFILES`).

\* **Columns**: `snake\_case` lowercase nouns (e.g. `user\_id`, `created\_at`).

\* **Primary Keys**: Always `ID` (except for map join tables).

\* **Foreign Keys**: `singular\_parent\_table\_id` (e.g. `user\_id`, `order\_id`).

\* **Constraints Prefix Coding**:

- Primary Keys: `PK\_` (e.g. `PK\_USERS`)

- Foreign Keys: `FK\_` (e.g. `FK\_CUST\_USER`)

- Unique Keys: `UK\_` (e.g. `UK\_USERS\_EMAIL`)

- Check Constraints: `CK\_` (e.g. `CK\_USERS\_STATUS`)

- Indexes: `IX\_` (e.g. `IX\_SESS\_USER`)

- Sequences: `SEQ\_` (e.g. `SEQ\_USERS`)

---

## SECTION 13: REST API MAPPING

The database tables map directly to future Spring Boot REST API routes:

|  | Relational Table         | REST API Endpoint             | Allowed Methods                  | Access Role               |
|--|--------------------------|-------------------------------|----------------------------------|---------------------------|
|  | ---                      | ---                           | ---                              | ---                       |
|  | `USERS`, `USER_SESSIONS` | `/api/v1/auth`                | `POST` (Login, Register, Logout) | Public / User             |
|  | `CUSTOMER_PROFILES`      | `/api/v1/customers`           | `GET`, `PUT`                     | Customer, Admin           |
|  | `VEHICLES`               | `/api/v1/customers/vehicles`  | `GET`, `POST`, `DELETE`          | Customer                  |
|  | `PROVIDER_PROFILES`      | `/api/v1/providers`           | `GET`, `PUT`                     | Provider, Admin           |
|  | `PROVIDER_DOCUMENTS`     | `/api/v1/providers/docs`      | `GET`, `POST`                    | Provider, Admin           |
|  | `ORDERS`                 | `/api/v1/orders`              | `GET`, `POST`, `PUT`             | Customer, Provider, Admin |
|  | `PAYMENTS`               | `/api/v1/payments`            | `POST` (Pay), `GET`              | Customer, Admin           |
|  | `FEEDBACK`               | `/api/v1/feedback`            | `POST` (Submit), `GET`           | Customer, Provider, Admin |
|  | `COLLABORATORS`          | `/api/v1/admin/collaborators` | `GET`, `POST`, `PUT`, `DELETE`   | Admin                     |
|  | `INQUIRIES`              | `/api/v1/contact-center`      | `POST` (Submit), `GET`, `PUT`    | Public, Admin             |
|  | `SERVICE_AREAS`          | `/api/v1/admin/service-areas` | `GET`, `POST`, `PUT`, `DELETE`   | Admin                     |

---

## SECTION 14: BACKEND LAYERS MAPPING

All Spring Boot services will follow this standard architectural mapping:

---

Database Table =====> Spring Boot Entity =====> JPA Repository =====> Business Service =====>  
REST Controller

"ORDERS" =====> "OrderEntity.java" =====> "OrderRepository" =====> "OrderService" =====>  
"OrderController"

---

## Reference Mapping Grid

### 1. \*\*Authentication\*\*:

\* `USERS` & rarr; `UserEntity` & rarr; `UserRepository` & rarr; `UserService` & rarr; `AuthController`

\* `USER\_SESSIONS` & rarr; `SessionEntity` & rarr; `SessionRepository` & rarr; `SessionService` & rarr; `AuthController`

### 2. \*\*Customer\*\*:

\* `CUSTOMER\_PROFILES` & rarr; `CustomerProfileEntity` & rarr; `CustomerProfileRepository` & rarr; `CustomerService` & rarr; `CustomerController`

\* `VEHICLES` & rarr; `VehicleEntity` & rarr; `VehicleRepository` & rarr; `CustomerService` & rarr; `CustomerController`

### 3. \*\*Provider\*\*:

\* `PROVIDER\_PROFILES` & rarr; `ProviderProfileEntity` & rarr; `ProviderProfileRepository` & rarr; `ProviderService` & rarr; `ProviderController`

### 4. \*\*Orders\*\*:

```

* `ORDERS` → `OrderEntity` → `OrderRepository` → `OrderService` →
`OrderController`

5. **Payments**:

* `PAYMENTS` → `PaymentEntity` → `PaymentRepository` → `PaymentService` →
`PaymentController`

6. **Feedback**:

* `FEEDBACK` → `FeedbackEntity` → `FeedbackRepository` → `FeedbackService`
→ `FeedbackController`

```

## SECTION 15: DATABASE LEARNING GUIDE

Below is a step-by-step trace of how the databases record system activities:

### 1. What happens in the database when a customer requests fuel?

- \*\*JPA Session Starts\*\***: Spring Boot backend validates the customer's session token against the `USER\_SESSIONS` table in the Oracle Database.
- \*\*Order Placement\*\***: The system inserts a new row into the `ORDERS` table (status `PENDING`). This database write triggers an insert in the `ORDER\_STATUS\_HISTORY` table (status `PENDING`, changed\_by `CUSTOMER`).
- \*\*Driver Search\*\***: Spring Boot queries MongoDB's `provider\_locations` collection using a `\$nearSphere` search to find active, available providers within an 8km radius.
- \*\*Driver Assigned\*\***: Once a provider is paired, Spring Boot updates `ORDERS.PROVIDER\_ID` with the driver's ID in Oracle. An assignment row is added to the `ORDER\_ASSIGNMENTS` table (status `PENDING`).
- \*\*Timeline Initialized\*\***: The backend creates an tracking document in MongoDB's `order\_tracking` collection, adding a timeline event `Request Created`.

### 2. What happens when a provider accepts an order?

- \*\*Update Assignment\*\***: Spring Boot updates the `ORDER\_ASSIGNMENTS.STATUS` column to `ACCEPTED` in Oracle.
- \*\*Update Order State\*\***: The system updates `ORDERS.STATUS` to `ACTIVE` in Oracle.
- \*\*Timeline Appended\*\***: The system adds an event `Provider Dispatched` to the timeline array inside the MongoDB `order\_tracking` document.
- \*\*Driver Status Update\*\***: The driver's availability status is updated to `ON\_JOB` in `PROVIDER\_AVAILABILITY` (Oracle) and `provider\_locations` (MongoDB).

### 3. What happens when payment is completed?

- \*\*Log Invoice State\*\***: The system updates `ORDERS.STATUS` to `COMPLETED` in Oracle.
- \*\*Clear Payment Invoice\*\***: The system updates `PAYMENTS.STATUS` to `SUCCESSFUL` and `PAYMENT\_TRANSACTIONS.STATUS` to `SUCCESSFUL` in Oracle.
- \*\*Log Revenue Split\*\***: Spring Boot calculates the revenue split and inserts a row into the `COMPANY\_LEDGER` table, logging the 20% corporate commission and 80% driver payout.



4. **\*\*Provider Earnings Updated\*\***: The system increments `PROVIDER_EARNINGS.PENDING_AMOUNT`` by the driver's payout amount in Oracle.
5. **\*\*Timeline Event Logged\*\***: The timeline event ``Payment Success`` is appended to MongoDB's `order_tracking`` collection.

## 4. What happens when feedback is submitted?

1. **\*\*Insert Review Row\*\***: The system inserts a row into the `FEEDBACK`` table (status ``New``) in Oracle.
2. **\*\*Flag Low Ratings\*\***: If the rating is  $\leq 2$  Stars, the status is set to ``Escalated`` to notify administrators.
3. **\*\*Recalculate Ratings\*\***: Spring Boot recalculates the provider's average rating and updates `PROVIDER_PROFILES.RATING`` and `PROVIDER_PROFILES.COMPLETED_DELIVERIES`` (increments by 1) in Oracle.

---

## SECTION 16: SCALABILITY STRATEGY

As user volumes scale, the database architecture grows using the following tiers:

### 1. 100 Active Users (Development / Staging)

\* **\*\*Configuration\*\***: Standalone database instances. Oracle Database Express Edition (XE) and MongoDB Community Edition running on a single development server.

\* **\*\*Tuning\*\***: Standard indexes on primary and foreign keys are sufficient. No partitioning or sharding required.

### 2. 1,000 Active Users (Local Launch)

\* **\*\*Configuration\*\***: Dedicated databases. Oracle Database Standard Edition and MongoDB Atlas (M30 tier).

\* **\*\*Tuning\*\***: Enable Spring Boot connection pooling (HikariCP). Configure automated nightly backups.

### 3. 10,000 Active Users (Regional Expansion)

\* **\*\*Configuration\*\***: High-availability clustering. Oracle Database running on a 2-node Real Application Clusters (RAC) setup. MongoDB configured as a 3-node replica set.

\* **\*\*Tuning\*\***: Apply range partitioning to the `AUDIT_LOGS`` and `ORDER_STATUS_HISTORY`` tables in Oracle. Set up read-replica query routing in Spring Boot to direct reports to secondary nodes.

### 4. 100,000 Active Users (National Scale)

\* **\*\*Configuration\*\***: Scaled sharding. Oracle Database Enterprise Edition with Active Data Guard replication. MongoDB configured with a sharded cluster.

\* **\*\*Tuning\*\***:

- **Oracle Partitioning**: Partition the `ORDERS` and `PAYMENTS` tables by date. Establish a data archiving job to move completed orders older than 180 days to a historical data warehouse.
- **MongoDB Sharding**: Shard the `system\_activity\_logs` and `provider\_locations` collections across a 3-shard cluster using hashed sharding keys.